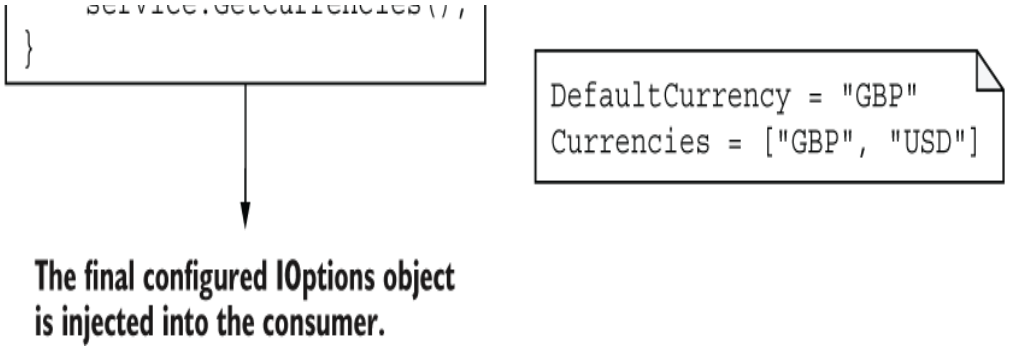


Figure 31.6 Using `OptionsBuilder` to build an `IOptions` object. Dependencies that are requested via the `Configure<TDeps>` methods are automatically retrieved from the DI container and used to execute the lambda function.

Using this pattern, we can update the code from listing 31.13 to use the `ICurrencyProvider` whenever our app needs to create the `CurrencyOptions` object. We can register the service in the DI container and know that the DI will take care of providing it to the lambda function at runtime, as shown in the following listing.

Listing 31.14 Using a DI service

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddOptions<CurrencyOptions>()
    .BindConfiguration("Currencies")
    .Configure<ICurrencyProvider>((opts, service) =>                    ❶
        opts.Currencies = service.GetCurrencies());                     ❶

builder.Services.AddSingleton<ICurrencyProvider, CurrencyProvider>();   ❷

WebApplication app = builder.Build();
app.MapGet("/", (IOptions<CurrencyOptions> opts) => opts.Value);        ❸
app.Run();
```

❶ Configures the Ioptions object using a service from DI

❷ Registers the service with the DI container